CSCE 548 — Project 4

Deployment & System Test Document

Movie Watchlist Application. Copyright Tsbih Salman

1. Project Overview

This document provides complete deployment instructions and full system test results for the CSCE 548 Movie Watchlist application. The application follows a 4-tier architecture: MySQL database, Python data access layer (DAL), FastAPI service layer, and an HTML/JavaScript frontend client.

The project was built with assistance from Claude (Anthropic), an AI language model, as part of an exercise in AI-assisted software development.

1.1 Architecture Summary

Layer	Technology / Details
Database	MySQL 8 — csce548_watchlist
Data Layer (DAL)	Python — movies_dao.py, lists_dao.py
Business Layer	Python — business.py (validation rules)
Service Layer	FastAPI + Uvicorn — service.py (port 8000)
Client Layer	HTML + JavaScript — index.html (port 5500)

1.2 Repository Structure

```
CSCE548/
├── project2/src/          # Backend: service, business, DAL, db
│   └── dal/               # movies_dao.py, lists_dao.py
├── project3/frontend/     # Frontend: index.html, app.js
├── sql/                   # 01_schema.sql, 02_seed.sql
└── README.md
```

---

2. Prerequisites

Install all of the following before proceeding:

Prerequisite	Version / Notes
Python	3.9 or higher (python3 --version)
pip	Bundled with Python
MySQL Community Server	8.0 or higher
Git	For cloning the repo
Web browser	Chrome, Firefox, or Safari

3. Step-by-Step Deployment

Step 1 — Clone the Repository

1. Open a terminal and navigate to your desired folder.
2. Clone the repo:

```
git clone https://github.com/2teaS/CSCE548.git
```

3. Enter the project folder: `cd CSCE548` **Step 2 — Start MySQL**

macOS (Homebrew):

```
brew services start mysql
```

macOS (official installer):

```
sudo /usr/local/mysql/support-files/mysql.server start
```

Linux:

```
sudo systemctl start mysql
```

Connect to verify:

```
mysql -u root -p
```

Step 3 — Set Up the Database

Inside the MySQL shell, run both SQL scripts:

```
SOURCE /full/path/to/CSCE548/sql/01_schema.sql;
SOURCE /full/path/to/CSCE548/sql/02_seed.sql;
USE csce548_watchlist;  SHOW TABLES;
```

Type EXIT; when done.

Step 4 — Configure the Database Connection

Open project2/src/db.py and update credentials to match your local MySQL:

```
password = 'YourPasswordHere'
```

Note: remove the unix_socket line if you encounter connection errors on macOS.

Step 5 — Install Python Dependencies

```
cd CSCE548/project2/src pip
install -r requirements.txt
```

Step 6 — Start the Backend Service

```
uvicorn service:app --reload --port 8000
```

You should see: Uvicorn running on http://127.0.0.1:8000
Leave this terminal open. Verify the API at: http://127.0.0.1:8000/docs

Step 7 — Serve the Frontend

Open a second terminal:

```
cd CSCE548/project3/frontend
python -m http.server 5500
```

Then open: http://127.0.0.1:5500/index.html
Click Ping — the status indicator should turn green (connected).

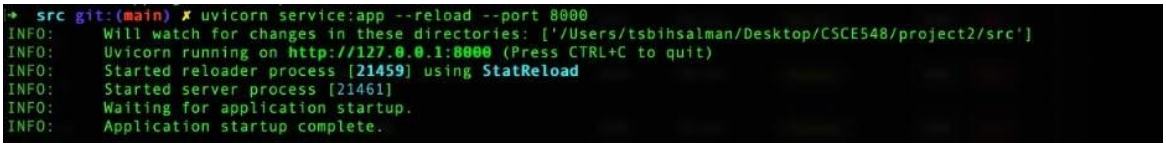

Figure 1 — Uvicorn backend started on port 8000

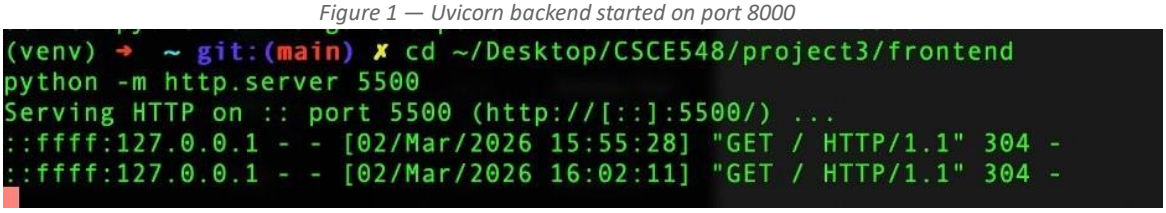

Figure 2 — Python HTTP server serving frontend on port 5500

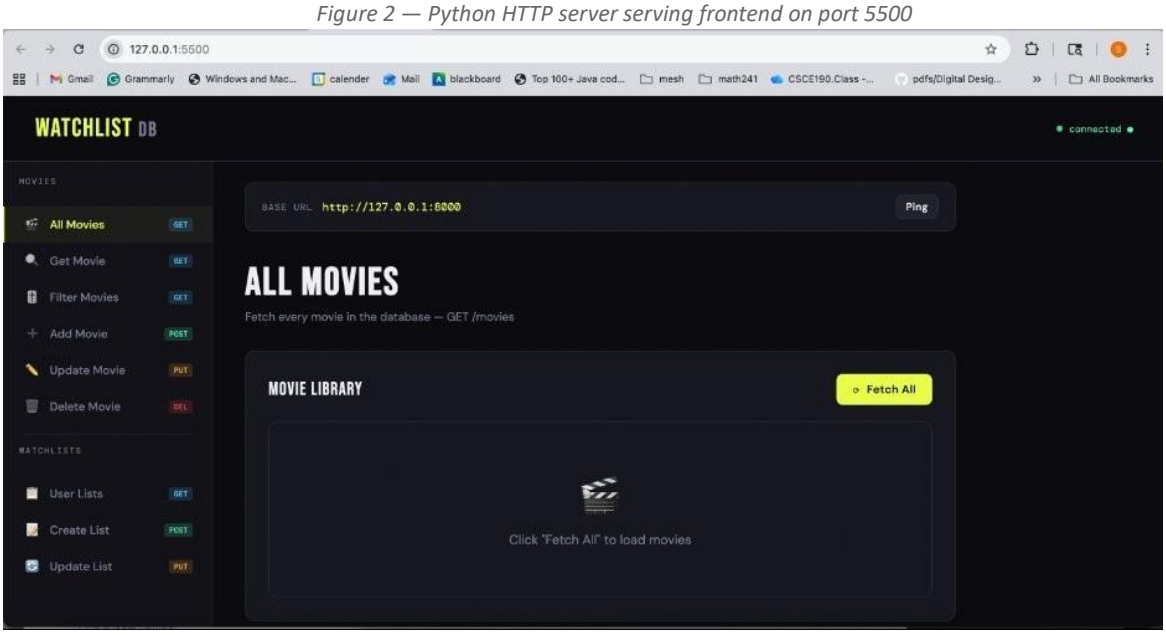

Figure 3 — Frontend loaded in browser, status: connected

Figure 4 — FastAPI auto-generated /docs showing all endpoints

4. Full System Test

Each CRUD operation was tested through both the browser frontend and the Python console client (client.py). Database state was verified in the MySQL terminal after each operation.

Test 1 — GET /movies (List All Movies)

Action: Clicked 'Fetch All' on the All Movies page. The movie table populated with all records from the database.

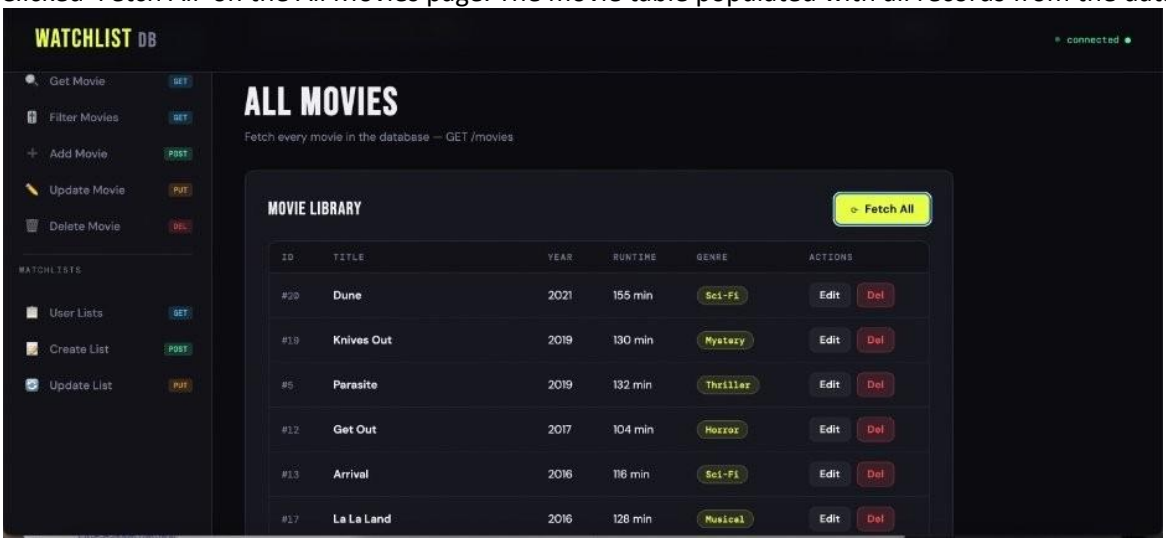

Figure 5 — Frontend: All Movies table loaded via GET /movies

Figure 6 — Console client: List movies (option 1) returns all records

Test 2 — GET /movies/{id} (Get Single Movie)

Action: Entered Movie ID 19 on the Get Movie page and clicked Fetch. The movie card displayed title 'Knives Out', year 2019, genre Mystery.

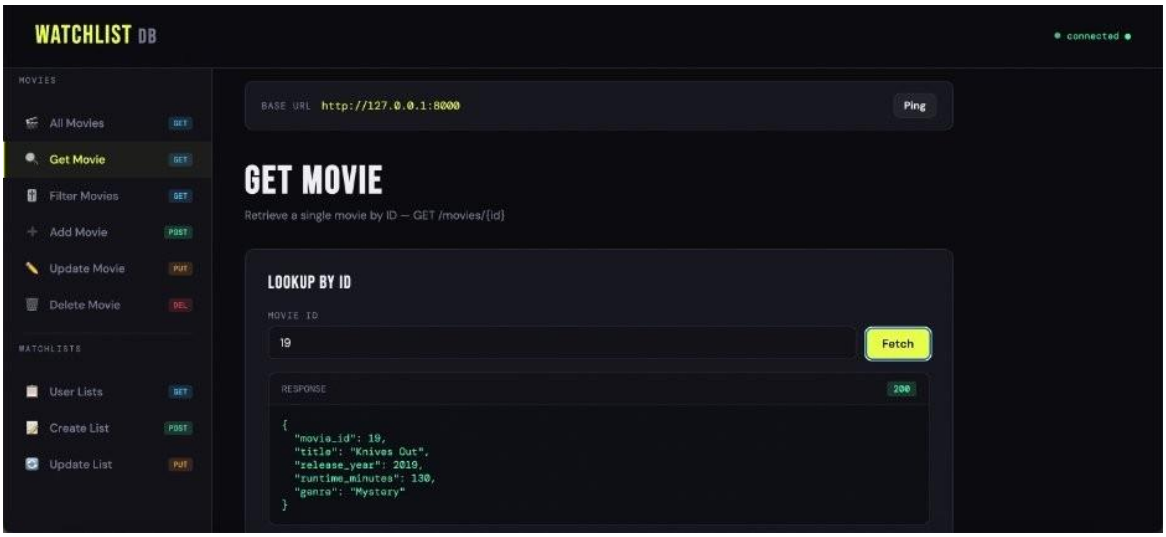

Figure 7 — Frontend: Get Movie by ID #19 returns Knives Out

Test 3 — POST /movies (Create Movie)

Action (Frontend): Filled the Add Movie form with title, year, runtime, and genre. Clicked Create Movie. The response confirmed movie_id 28 was assigned.

Action (Console client): Created movie 'Wuthering Heights' (2026, 136 min, romance) via option 2. Response confirmed movie_id 21 with read-back.

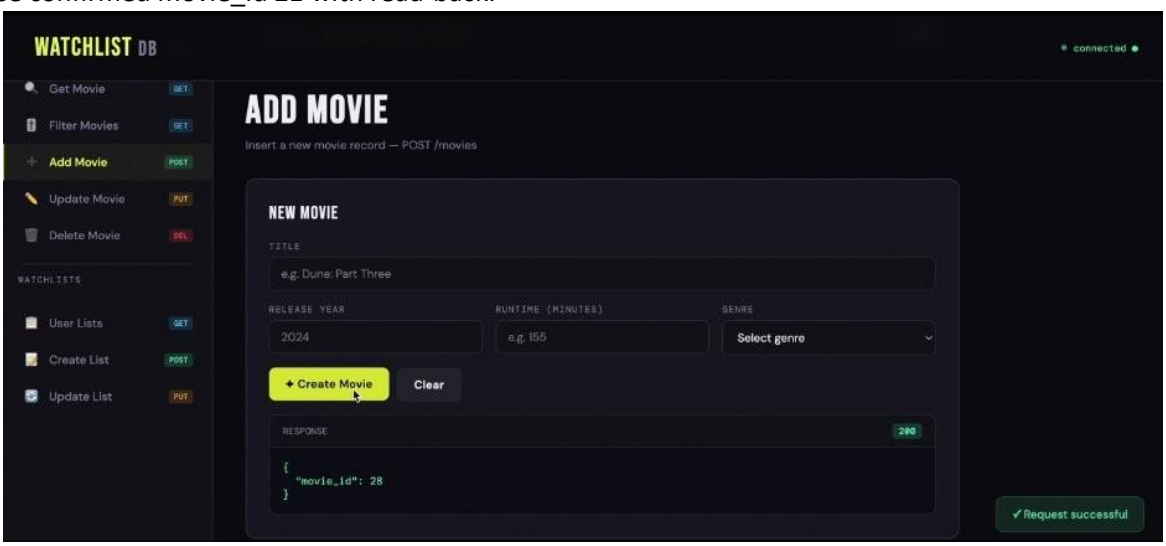

Figure 8 — Frontend: Create Movie response shows movie_id: 28

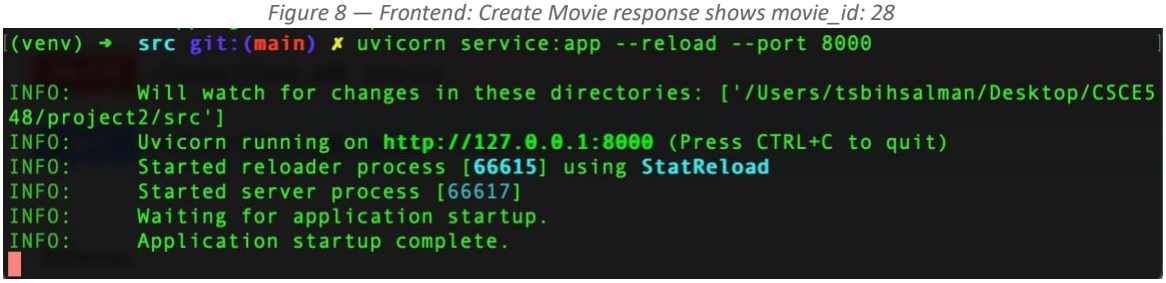

Figure 9 — Console client: Create movie returns movie_id 21 with read-back

Test 4 — PUT /movies/{id} (Update Movie)

Action (Frontend): Loaded Movie #20, changed the release year to 2009 and runtime to 156. Clicked Save Changes. Response: {updated: true}.

Action (Console client): Loaded Movie #21 (Wuthering Heights) and initiated update via option 4.

Database verification: SELECT * FROM movies WHERE movie_id = 20 confirms the update — Dune now shows release_year 2009 and runtime_minutes 156.

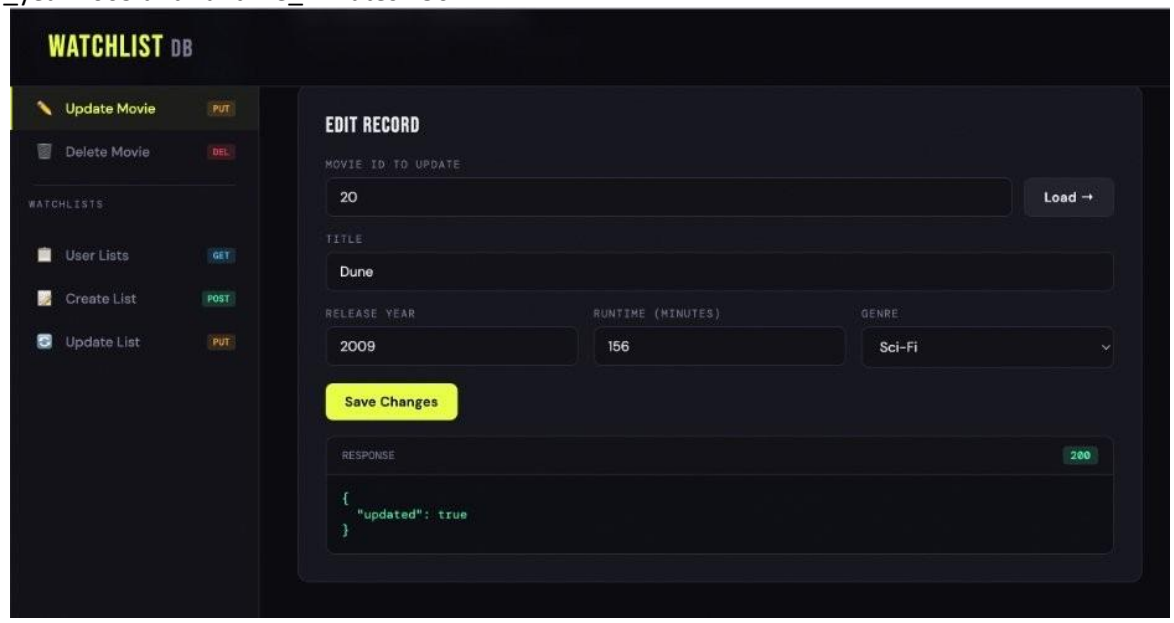

Figure 10 — Frontend: Update Movie #20 response shows updated: true

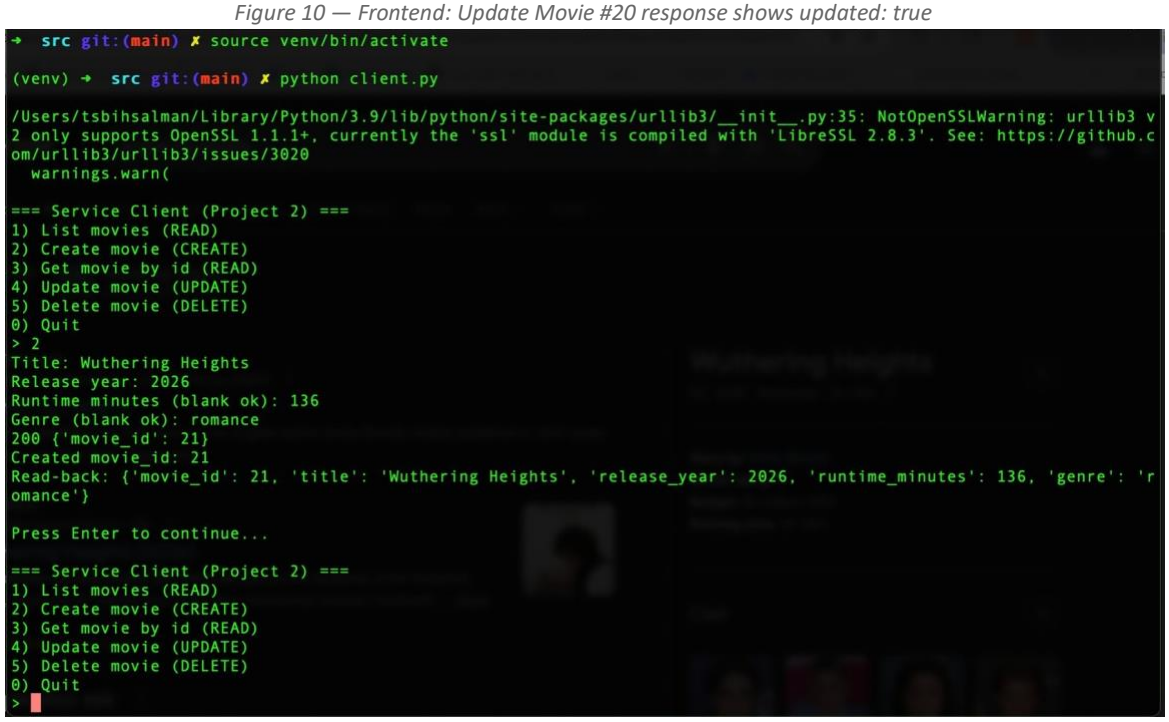

Figure 11 — Console client: Update movie #21 loaded for editing

Test 5 — DELETE /movies/{id} (Delete Movie)

Action (Frontend): Entered Movie ID 28 on the Delete Movie page. Confirmed the deletion. Response: {deleted: true}.

Action (Console client): Deleted Movie #21 via option 5. Confirmed with YES. Response: 200 {deleted: True}. Read-back returned 404 Movie not found, confirming successful deletion.

Database verification: SELECT * FROM movies WHERE movie_id = 28 returns Empty set, confirming the record was removed.

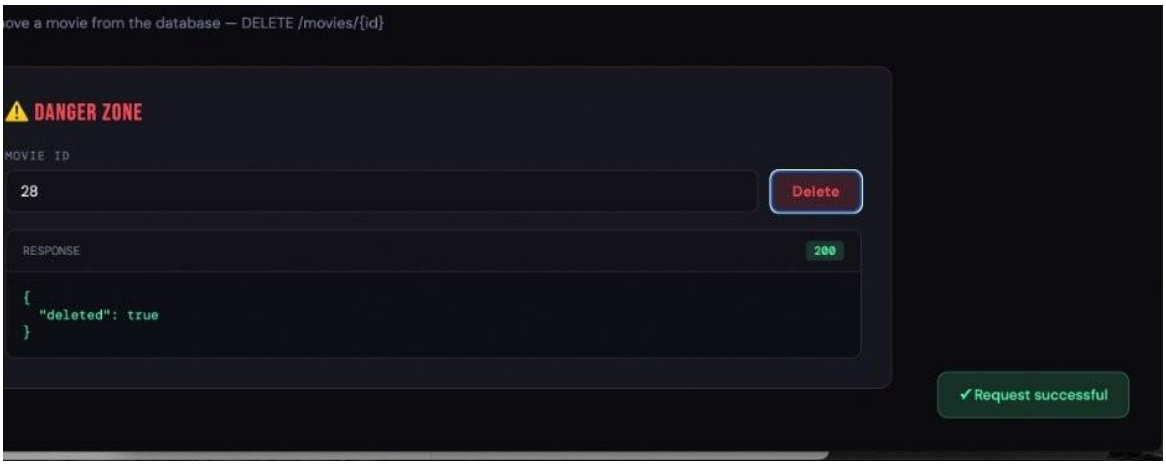

Figure 12 — Frontend: Delete Movie #28 response shows deleted: true

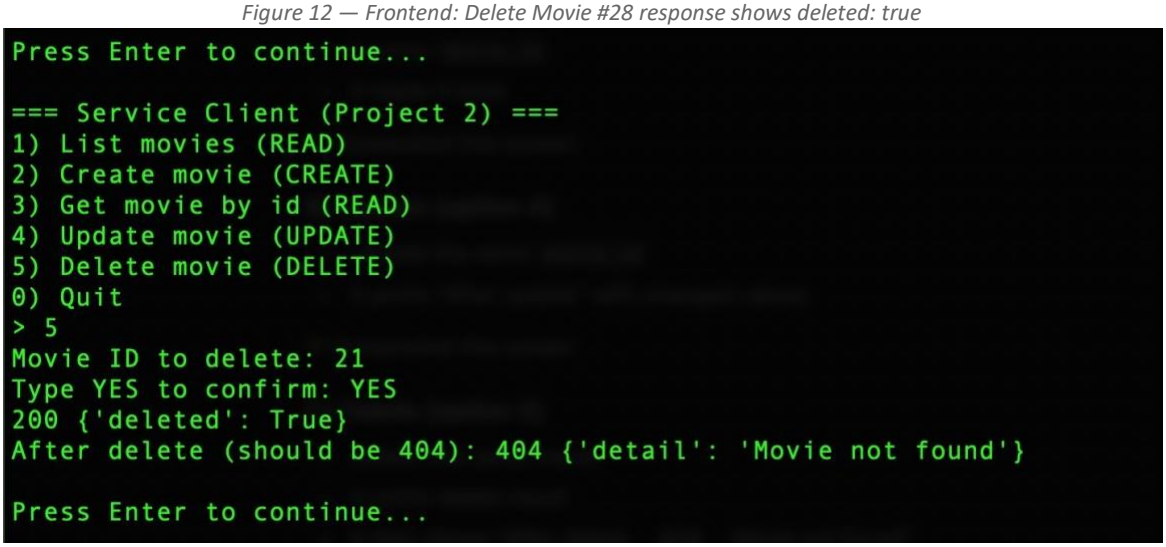

Figure 13 — Console client: Delete movie #21 — 200 success, 404 on read-back

Database Verification — MySQL Terminal

The following MySQL queries confirm the results of all four write operations:

- SELECT * FROM movies LIMIT 5 — confirms seeded data is present
- SELECT * FROM movies ORDER BY movie_id DESC LIMIT 1 — shows most recently inserted record
- SELECT * FROM movies WHERE movie_id = 20 — confirms update: Dune, 2009, 156 min, SciFi
- SELECT * FROM movies WHERE movie_id = 28 — returns Empty set, confirming deletion

```
mysql> SELECT * FROM movies LIMIT 5;
ERROR 1046 (3D000): No database selected
mysql> USE csce548_watchlist;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM movies LIMIT 5;
+----------+-----------------+--------------+-----------------+----------+
| movie_id | title           | release_year | runtime_minutes | genre    |
+----------+-----------------+--------------+-----------------+----------+
|        1 | The Matrix      |         1999 |             136 | Sci-Fi   |
|        2 | Inception       |         2010 |             148 | Sci-Fi   |
|        3 | Interstellar    |         2014 |             169 | Sci-Fi   |
|        4 | The Dark Knight |         2008 |             152 | Action   |
|        5 | Parasite        |         2019 |             132 | Thriller |
+----------+-----------------+--------------+-----------------+----------+
5 rows in set (0.001 sec)

mysql> SELECT * FROM movies ORDER BY movie_id DESC LIMIT 1;
+----------+-------+--------------+-----------------+----------+
| movie_id | title | release_year | runtime_minutes | genre    |
+----------+-------+--------------+-----------------+----------+
|       27 | Raazi |         2018 |             155 | Thriller |
+----------+-------+--------------+-----------------+----------+
1 row in set (0.003 sec)

mysql> SELECT * FROM movies WHERE movie_id = 20;
+----------+-------+--------------+-----------------+--------+
| movie_id | title | release_year | runtime_minutes | genre  |
+----------+-------+--------------+-----------------+--------+
|       20 | Dune  |         2009 |             156 | Sci-Fi |
+----------+-------+--------------+-----------------+--------+
1 row in set (0.007 sec)

mysql> SELECT * FROM movies WHERE movie_id = 28;
Empty set (0.003 sec)

mysql>
```

Figure 14 — MySQL terminal: all four DB verification queries in one session

---

5. Graduate Credit — AI Tool Analysis

5.1 Prompts Used to Generate the Code

The following prompts were used to generate the application, organized by layer:

Database Layer (SQL schema and seed data)

Prompt: "Design a MySQL schema for a movie watchlist application. Include tables for movies, users, watchlists, and list items. Generate a schema SQL file and a seed file with at least 50 rows of realistic test data."

Data Access Layer (DAL)

Prompt: "Write a Python data access layer using mysql-connector-python for the movie watchlist schema. Create two DAO classes: MoviesDAO and ListsDAO. Each should implement full CRUD — create, get_by_id, list_all, update, and delete. Use parameterized queries."

Business Layer

Prompt: "Create a Python business layer class called BusinessLayer that wraps the DAL. Add input validation: movie titles cannot be empty, release_year must be between 1888 and 2100, runtime_minutes must be positive if provided, and list names cannot be empty. Raise ValueError for violations."

Service Layer

Prompt: "Build a FastAPI REST API service that uses the BusinessLayer. Expose endpoints for full CRUD on movies (GET /movies, GET /movies/{id}, POST /movies, PUT /movies/{id}, DELETE /movies/{id}) and CRUD on watchlists. Return appropriate HTTP status codes and error messages."

Frontend (Client Layer)

Prompt: "Build a dark-themed HTML/CSS/JavaScript single-page frontend for the movie watchlist API. Include a sidebar navigation, forms for all CRUD operations, a movie table with Edit and Delete quickaction buttons, and a toast notification system. Do not display raw JSON — show user-friendly messages and formatted cards instead."

5.2 Changes Made to AI-Generated Output

- db.py — Added unix_socket parameter for macOS compatibility with locally installed MySQL.
- service.py — Added CORS middleware (CORSMiddleware) to allow the frontend served on port 5500 to communicate with the API on port 8000. This was not included in the initial generation.
- app.js — Fixed API_BASE port from 8080 to 8000 to match the uvicorn server port.
- business.py imports — Corrected from absolute imports (from movies_dao import ...) to package-relative imports (from dal.movies_dao import ...) to match the actual directory structure.
- index.html showResponse() — The original frontend displayed raw JSON output after every API call. This was replaced with human-readable UI cards: movie detail cards, confirmation messages, and friendly error states. This change was made in response to instructor feedback after Project 3.

5.3 Effectiveness Analysis

The AI tool (Claude by Anthropic) was highly effective at generating the structural boilerplate of each layer. The overall 4-tier architecture, data models, SQL schema, DAO patterns, FastAPI route definitions, and frontend layout were all generated correctly on the first or second prompt.

Areas where AI performed well:

- Generated all five REST endpoints with correct HTTP methods and status codes on the first attempt.
- Business layer validation rules (year range, required fields, positive runtime) were complete and correct.
- DAL parameterized queries were syntactically correct and safe against SQL injection.
- Frontend layout, dark theme, and sidebar navigation required minimal revision.

Areas where AI fell short:

- Environment-specific configuration: The AI did not account for macOS unix socket vs TCP connection differences in MySQL, requiring a manual fix to db.py.
- CORS configuration: Cross-Origin Resource Sharing middleware was omitted from the initial service layer generation, causing the frontend to fail silently until manually added.
- Port consistency: The frontend and backend were generated with different port numbers (8080 vs 8000), requiring manual correction.
- UX quality: The initial frontend used raw JSON dumps as user feedback, which is technically correct but not user-friendly. The AI required an explicit follow-up prompt to replace this with formatted UI components. The professor specifically flagged this in Project 3 feedback.

Overall assessment:

AI tools are excellent at generating syntactically correct, architecturally sound code. However, they require developer oversight for environment-specific details, cross-component consistency (ports, import paths), and UX quality. The AI saved approximately 70-80% of initial coding time, while the remaining effort was spent on integration, debugging, and polish.

---

6. Troubleshooting

Problem	Solution

Frontend shows 'Server unreachable'	Start uvicorn (Step 6). Confirm baseUrl in UI is port 8000.
MySQL connection error in uvicorn logs	Check db.py credentials. Remove unix_socket line on macOS if needed.
ModuleNotFoundError on startup	Run pip install -r requirements.txt from project2/src/.
CORS error in browser console	Serve frontend via http:// not file://. Use python -m http.server 5500.
Port 8000 already in use	Run: lsof -ti:8000 | xargs kill -9, then restart uvicorn.